





Uso de librerías de interfaz gráfica

Propiedades de los componentes de interfaz gráfica



La librería *Swing* brinda:

- Una serie de componentes gráficos (JButton, JTextArea, JTextField, etc.) que puedes usar para crear una interfaz gráfica.
- Estos componentes se pueden personalizar, hasta cierto punto.

A continuación, se muestran algunos componentes con sus respectivos métodos:

JButton

JButton

JTextField

JButton

JTextField

JTextField



Uso de librerías de interfaz gráfica

Ejemplo 1



Observa el siguiente ejemplo y lee los pasos que se describen en los cuadros:



Si deseas conocer más acerca de los constructores y de los métodos que proveen las clases de la librería Swing, te sugiero revisar la siguiente información:





Uso de librerías de interfaz gráfica

Ejemplo 1



Observa el siguiente ejemplo y lee los pasos que se describen en los cuadros:

Para aprender más:

Enlace: <https://docs.oracle.com/javase/7/docs/api/javax/swing/JOptionPane.html>

Autor: Oracle

Título: Plataforma Java, edición estándar 7

Especificación API

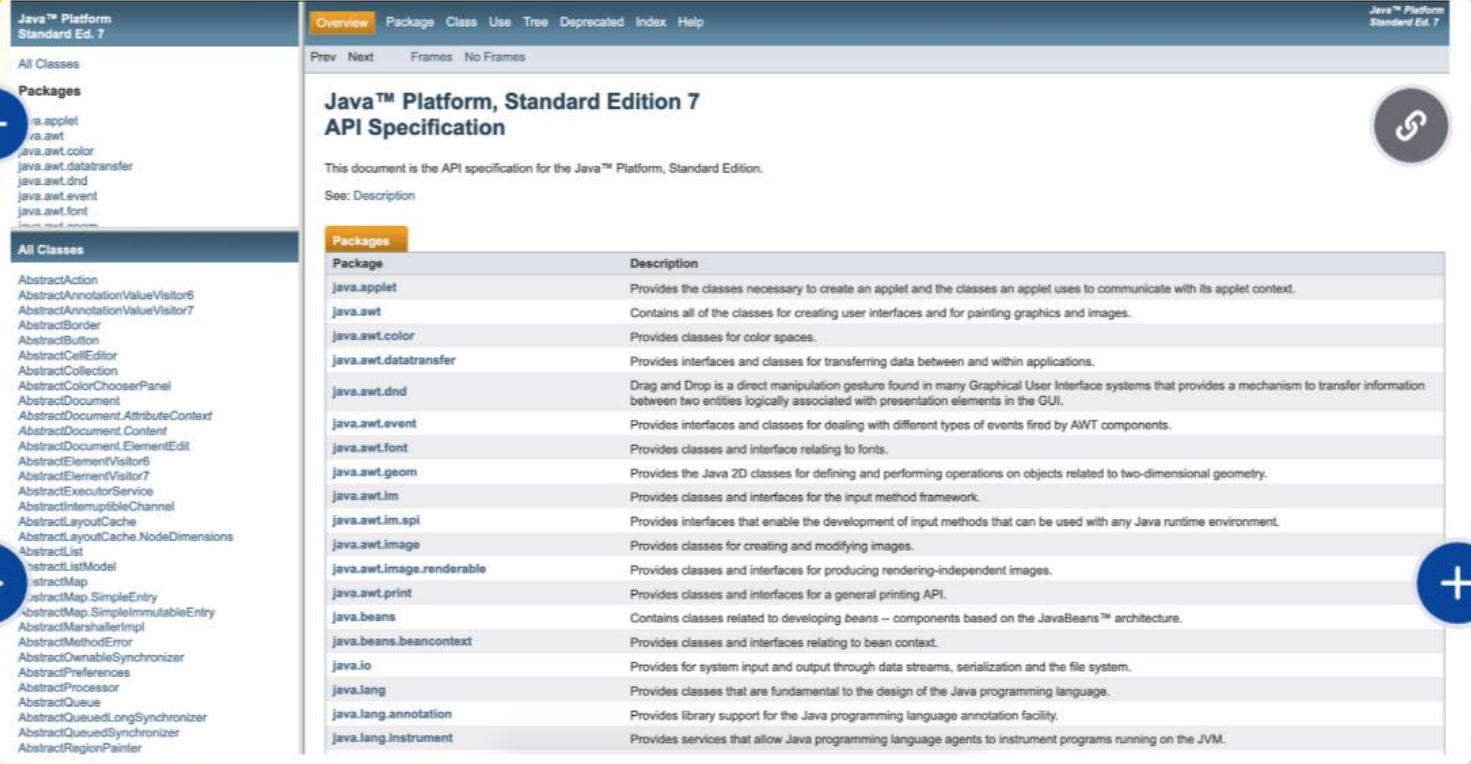
y de los métodos que proveen las clases de la librería Swing, te sugiero revisar la siguiente información:



Uso de librerías de interfaz gráfica

Ejemplo 1





Java™ Platform, Standard Edition 7: <https://docs.oracle.com/javase/7/docs/api/>



Uso de librerías de interfaz gráfica

Ejemplo 1

Java™ Platform
Standard Ed. 7

Overview Package Class Use Tree Deprecated Index Help

Prev Next Frames No Frames

Java™ Platform, Standard Edition 7 API Specification

This document is the API specification for the Java™ Platform, Standard Edition.

See: Description

All Classes

Packages

- java.applet
- java.awt
- java.awt.color
- java.awt.datatransfer
- java.awt.dnd
- java.awt.event
- java.awt.font
- java.awt.geom
- java.awt.im
- java.awt.im.spi
- java.awt.image
- java.awt.image.renderable
- java.awt.print
- java.beans
- java.beans.beancontext
- java.io
- java.lang
- java.lang.annotation
- java.lang.instrument

All Classes

- AbstractAction
- AbstractAnnotation
- AbstractBorder
- AbstractButton
- AbstractCellEditor
- AbstractCollection
- AbstractColorChooserPanel
- AbstractDocument
- AbstractDocument.AttributeContext
- AbstractDocument.Content
- AbstractDocument.ElementEdit
- AbstractElementVisitor
- AbstractElementVisitor7
- AbstractExecutorService
- AbstractInterruptibleChannel
- AbstractLayoutCache
- AbstractLayoutCache.NodeDimensions
- AbstractList
- AbstractListModel
- AbstractMap
- AbstractMap.SimpleEntry
- AbstractMap.SimpleImmutableEntry
- AbstractMarshallerImpl
- AbstractMethodError
- AbstractOwnableSynchronizer
- AbstractPreferences
- AbstractProcessor
- AbstractQueue
- AbstractQueuedLongSynchronizer
- AbstractQueuedSynchronizer
- AbstractRegionPainter

Información acerca de cada elemento de los bloques de la izquierda



Uso de librerías de interfaz gráfica

Ejemplo 1

Java™ Platform
Standard Ed. 7

Overview Package Class Use Tree Deprecated Index Help

Prev Next Frames No Frames

Java™ Platform, Standard Edition 7 API Specification

This document is the API specification for the Java™ Platform, Standard Edition.

See: Description

All Classes

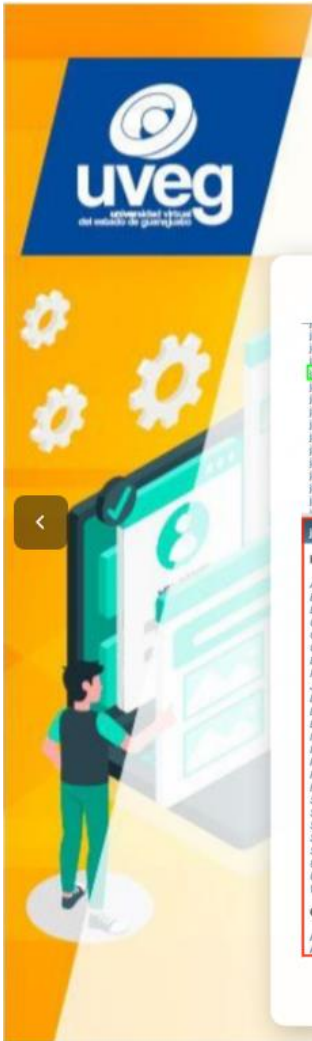
Packages

- java.applet
- java.awt
- java.awt.color
- java.awt.datatransfer
- java.awt.dnd
- java.awt.event
- java.awt.font
- java.awt.geom
- java.awt.im
- java.awt.im.spi
- java.awt.image
- java.awt.image.renderable
- java.awt.print
- java.beans
- java.beans.beancontext
- java.io
- java.lang
- java.lang.annotation
- java.lang.instrument

All Classes

- AbstractAction
- AbstractAnnotation
- AbstractBorder
- AbstractButton
- AbstractCellEditor
- AbstractCollection
- AbstractColorChooserPanel
- AbstractDocument
- AbstractDocument.AttributeContext
- AbstractDocument.Content
- AbstractDocument.ElementEdit
- AbstractElementVisitor
- AbstractElementVisitor7
- AbstractExecutorService
- AbstractInterruptibleChannel
- AbstractLayoutCache
- AbstractLayoutCache.NodeDimensions
- AbstractList
- AbstractListModel
- AbstractMap
- AbstractMap.SimpleEntry
- AbstractMap.SimpleImmutableEntry
- AbstractMarshallerImpl
- AbstractMethodError
- AbstractOwnableSynchronizer
- AbstractPreferences
- AbstractProcessor
- AbstractQueue
- AbstractQueuedLongSynchronizer
- AbstractQueuedSynchronizer
- AbstractRegionPainter

Paquetes que provee Java



Uso de librerías de interfaz gráfica

Ejemplo 2

[java.awt.event.ActionListener](#)

[javax.swing.border.Border](#)

[javax.swing.colorchooser.ColorChooser](#)

[javax.swing.event.EventListenerList](#)

[javax.swing.flexiblerenderer.FlexibleRenderer](#)

[javax.swing.plaf.basic.BasicLookAndFeel](#)

[javax.swing.plaf.metal.MetalLookAndFeel](#)

[javax.swing.plaf.multi.MultiLookAndFeel](#)

[javax.swing.plaf.nimbus.NimbusLookAndFeel](#)

[javax.swing.plaf.synth.SynthLookAndFeel](#)

[Overview](#) |
 [Package](#) |
 [Class](#) |
 [Use Tree](#) |
 [Deprecated](#) |
 [Index](#) |
 [Help](#)

[Prev](#) |
 [Next](#) |
 Frames: [No Frames](#)

Java™ Platform, Standard Edition 7 API Specification

This document is the API specification for the Java™ Platform, Standard Edition.

See: [Description](#)

java.swing

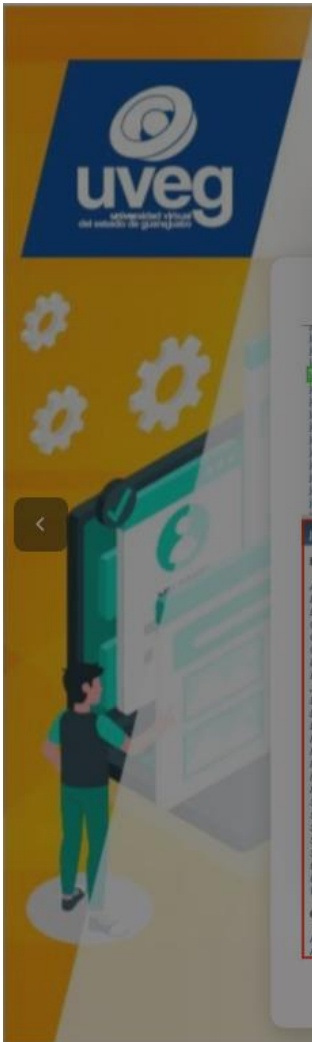
Interfaces

[Action](#)
[BoundedRangeModel](#)
[ButtonModel](#)
[CellEditor](#)
[ComboBoxEditor](#)
[ComboBoxModel](#)
[DesktopManager](#)
[Icon](#)
[JComboBox.KeySelectionManager](#)
[ListCellRenderer](#)
[ListModel](#)
[ListSelectionModel](#)
[MenuItem](#)
[MutableComboBoxModel](#)
[Painter](#)
[Renderer](#)
[RootPaneContainer](#)
[Scrollable](#)
[ScrollPaneConstants](#)
[SingleSelectionModel](#)
[SpinnerModel](#)
[SwingConstants](#)
[UIDefaults.ActiveValue](#)
[UIDefaults.LazyValue](#)
[WindowConstants](#)

Classes

[AbstractAction](#)
[AbstractButton](#)

Package	Description
java.applet	Provides the classes necessary to create an applet and the classes an applet uses to communicate with its applet context.
java.awt	Contains all of the classes for creating user interfaces and for painting graphics and images.
java.awt.color	Provides classes for color spaces.
java.awt.datatransfer	Provides interfaces and classes for transferring data between and within applications.
java.awt.dnd	Drag and Drop is a direct manipulation gesture found in many Graphical User Interface systems that provides a mechanism to transfer information between two entities logically associated with presentation elements in the GUI.
java.awt.event	Provides interfaces and classes for dealing with different types of events fired by AWT components.
java.awt.font	Provides classes and interface relating to fonts.
java.awt.geom	Provides the Java 2D classes for defining and performing operations on objects related to two-dimensional geometry.
java.awt.im	Provides classes and interfaces for the input method framework.
java.awt.im.spi	Provides interfaces that enable the development of input methods that can be used with any Java runtime environment.
java.awt.image	Provides classes for creating and modifying images.
java.awt.image.renderable	Provides classes and interfaces for producing rendering-independent images.
java.awt.print	Provides classes and interfaces for a general printing API.
java.beans	Contains classes related to developing beans – components based on the JavaBeans™ architecture.
java.beans.beancontext	Provides classes and interfaces relating to bean context.
java.io	Provides for system input and output through data streams, serialization and the file system.
java.lang	Provides classes that are fundamental to the design of the Java programming language.
java.lang.annotation	Provides library support for the Java programming language annotation facility.
java.lang.instrument	Provides services that allow Java programming language agents to instrument programs running on the JVM.



Uso de librerías de interfaz gráfica

Ejemplo 2

java.swing

- Action
- BoundedRangeModel
- ButtonModel
- CaretListener
- ComboBoxEditor
- ComboBoxModel
- DesktopManager
- Icon
- JComboBox.KeySelectionManager
- ListCellRenderer
- ListModel
- ListSelectionModel
- MenuItem
- MutableComboBoxModel
- Painter
- Renderable
- RolloverContainer
- Scrollable
- ScrollPaneConstants
- SingleSelectionModel
- SpinnerModel
- SwingConstants
- UIDefaults.ActiveValue
- UIDefaults.LazyValue
- WindowConstants

Classes

- AbstractAction
- AbstractButton

Java™ Platform, Standard Edition 7 API Specification

This document is the API specification for the Java™ Platform, Standard Edition.

See: Description

Package	Description
java.applet	Provides the classes necessary to create an applet and the classes an applet uses to communicate with its applet context.
java.awt	Contains all of the classes for creating user interfaces and for painting graphics and images.
java.awt.color	Provides classes for color spaces.
java.awt.datatransfer	Provides interfaces and classes for transferring data between and within applications.
java.awt.dnd	Drag and Drop is a direct manipulation gesture found in many Graphical User Interface systems that provides a mechanism to transfer information between two entities logically associated with presentation elements in the GUI.
java.awt.event	Provides interfaces and classes for dealing with different types of events fired by AWT components.
java.awt.font	Provides classes and interface relating to fonts.
java.awt.geom	Provide the Java 2D classes for defining and performing operations on objects related to two-dimensional geometry.
java.awt.im	Provides interfaces that enable the development of input methods that can be used with any Java runtime environment.
java.awt.image	Provides classes for creating and modifying images.
java.awt.image.renderable	Provides classes and interfaces for producing rendering-independent images.
java.awt.print	Provides classes and interfaces for a general printing API.
java.beans	Contains classes related to developing beans – components based on the JavaBeans™ architecture.
java.beans.beancontext	Provides classes and interfaces relating to bean context.
java.io	Provides for system input and output through data streams, serialization and the file system.
java.lang	Provides classes that are fundamental to the design of the Java programming language.
java.lang.annotation	Provides library support for the Java programming language annotation facility.
java.lang.instrument	Provides services that allow Java programming language agents to instrument programs running on the JVM.



Uso de librerías de interfaz gráfica

JButton

La clase JButton usa:

- La sobrecarga de constructores; es decir, cuenta con más de un constructor, los cuales tienen diferentes parámetros.

En la tabla se muestran cada uno de estos constructores:

Constructor	Descripción
JButton()	Crea un botón sin texto ni ícono.
JButton(Action action)	Crea un botón a partir de las propiedades que son definidas en la acción.
JButton(Icon icon)	Crea un botón con un ícono.
JButton(String text)	Crea un botón con un texto definido.
JButton(String text, Icon icon)	Crea un botón con un texto e ícono definido.



Uso de librerías de interfaz gráfica

JButton

La clase JButton usa:

- La sobrecarga de constructores; es decir, cuenta con más de un constructor, los cuales tienen diferentes parámetros.

En la tabla se muestran cada uno de estos constructores:

Constructor	Descripción
JButton()	Crea un botón sin texto ni ícono.
JButton(Action action)	Crea un botón a partir de las propiedades que son definidas en la acción.
JButton(Icon icon)	Crea un botón con un ícono.
JButton(String text)	Crea un botón con un texto definido.
JButton(String text, Icon icon)	Crea un botón con un texto e ícono definido.



Uso de librerías de interfaz gráfica

JButton

Observa en la tabla los **métodos que puedes usar** para agregar funcionalidad o personalizar el botón mediante la **instancia de la clase JButton**.

Método	Descripción
addActionListener	Agrega el objeto oyente para estar en escucha de los eventos que ocurren sobre el botón.
setText	Establece el texto del botón.
setHorizontalAlignment	Establece la alineación del texto en el eje horizontal.
setVerticalAlignment	Establece la alineación del texto en el eje vertical.
setBorderPainted	Define si el borde es pintado o no.
setEnabled	Habilita o deshabilita el botón por defecto es <i>true</i> .
getText	Obtiene el texto que fue definido en el botón.
setIcon	Establece un ícono para el botón.



Uso de librerías de interfaz gráfica

JButton

Observa en la tabla los **métodos que puedes usar** para agregar funcionalidad o personalizar el botón mediante la **instancia de la clase JButton**.

Método	Descripción
addActionListener	Agrega el objeto oyente para estar en escucha de los eventos que ocurren sobre el botón.
setText	Establece el texto del botón.
setHorizontalAlignment	Establece la alineación del texto en el eje horizontal.
setVerticalAlignment	Establece la alineación del texto en el eje vertical.
setBorderPainted	Define si el borde es pintado o no.
setEnabled	Habilita o deshabilita el botón por defecto es <i>true</i> .
getText	Obtiene el texto que fue definido en el botón.
setIcon	Establece un ícono para el botón.



Uso de librerías de interfaz gráfica

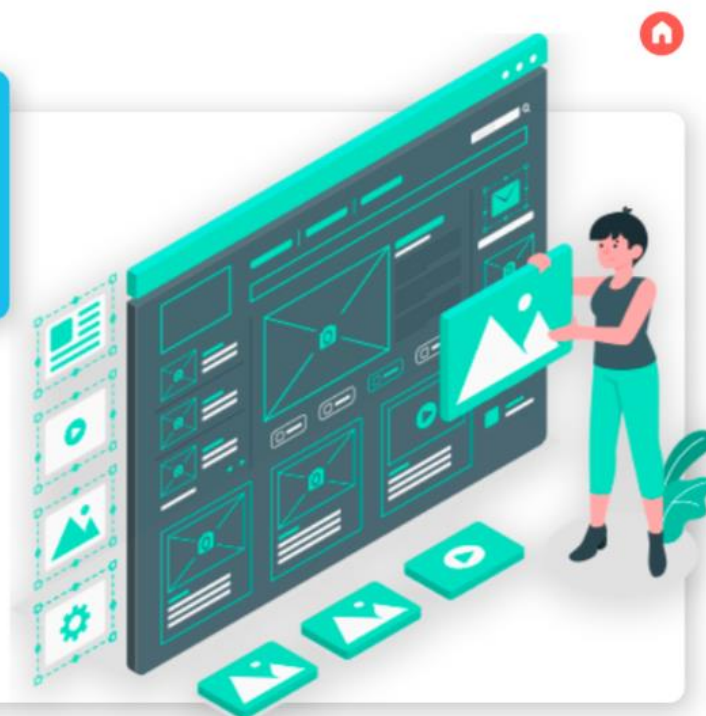
Manipulación de los componentes de interfaz gráfica

Los componentes de interfaz gráfica pueden ser manipulados por:

- Otros elementos de la misma naturaleza, o
- Mediante alguna condición que se ejecute en la aplicación.

Esos componentes son manipulados mediante los diferentes métodos que cada uno posee.

Por ejemplo: setSize, setBounds, setEnable, etcétera.



Uso de librerías de interfaz gráfica

Manipulación de los componentes de interfaz gráfica

A continuación, observa **tres ejemplos** en los que se usa la **manipulación de componentes de interfaz gráfica**:

Creación de una ventana



Modificación del título de una ventana



Creación de una ventana (que a través de un botón crea más botones)





Uso de librerías de interfaz gráfica

Creación de una ventana

Observa el siguiente ejemplo, donde se creó una ventana en la cual se modifican las dimensiones del **componente JTextArea**:

Manipulando componentes

Tamaño en X: 100
Tamaño en Y: 50

Manipulando componentes

¿Qué es Lorem Ipsum?
Lorem Ipsum es simplemente el texto de relleno de las imprentas y archivos de texto. Lorem Ipsum ha sido el texto de relleno estándar de las industrias desde el año 1500, cuando un impresor (N. del T. persona que se dedica a la imprenta) desconocido usó una galería de textos y los mezcló de tal manera que logró hacer un libro de textos especimen. No sólo sobrevivió 500 años, sino que también ingresó como texto de relleno en documentos electrónicos, quedando esencialmente igual al original. Fue popularizado en los 60s con la creación de las hojas "Letras", las cuales contenían pasajes de Lorem Ipsum, y más recientemente con software de autoedición, como por ejemplo Aldus PageMaker, el cual incluye versiones de Lorem Ipsum.

¿Por qué lo usamos?
Es un hecho establecido hace demasiado tiempo que un lector se distraerá con el contenido del texto de un sitio mientras que mira su diseño. El punto de usar Lorem Ipsum es que tiene una distribución más o menos normal de las letras, al contrario de usar textos como por ejemplo "Contenido aquí, contenido aquí". Estos textos hacen parecerlo un espacio que se puede leer. Muchos paquetes de autoedición y editores de páginas web usan el Lorem Ipsum como su texto por defecto, y al hacer una búsqueda de "Lorem Ipsum" va a dar por resultado muchos sitios.

Tamaño en X: 450
Tamaño en Y: 350



Uso de librerías de interfaz gráfica

Creación de una ventana

Observa el siguiente ejemplo, donde se creó una ventana en la cual se modifican las dimensiones del **componente JTextArea**:

Componente al que se le modifican las dimensiones

Manipulando componentes

Tamaño en X: 100
Tamaño en Y: 50

Manipulando componentes

ue se dedica a la imprenta) desconocido usó una galería de textos y los mezcló de tal manera que logró hacer un libro de textos especimen. No sólo sobrevivió 500 años, sino que también ingresó como texto de relleno en documentos electrónicos, quedando esencialmente igual al original. Fue popularizado en los 60s con la creación de las hojas "Letras", las cuales contenían pasajes de Lorem Ipsum, y más recientemente con software de autoedición, como por ejemplo Aldus PageMaker, el cual incluye versiones de Lorem Ipsum.

¿Por qué lo usamos?
Es un hecho establecido hace demasiado tiempo que un lector se distraerá con el contenido del texto de un sitio mientras que mira su diseño. El punto de usar Lorem Ipsum es que tiene una distribución más o menos normal de las letras, al contrario de usar textos como por ejemplo "Contenido aquí, contenido aquí". Estos textos hacen parecerlo un espacio que se puede leer. Muchos paquetes de autoedición y editores de páginas web usan el Lorem Ipsum como su texto por defecto, y al hacer una búsqueda de "Lorem Ipsum" va a dar por resultado muchos sitios.

Tamaño en X: 450
Tamaño en Y: 350



Uso de librerías de interfaz gráfica

Creación de una ventana

Observa el siguiente ejemplo, donde se creó una ventana en la cual se modifican las dimensiones del **componente JTextArea**:

Elementos que editan las dimensiones del componente JTextArea

Tamaño en X: 100

Tamaño en Y: 50

+

+

ue se dedica a la imprenta) desconocido usó una galería de textos y los mezcló de tal manera que logró hacer un libro de textos especimen. No sólo sobrevivió 500 años, sino que también ingresó como texto de relleno en documentos electrónicos, quedando esencialmente igual al original. Fue popularizado en los 60s con la creación de las hojas "Letrasas", las cuales contenían pasajes de Lorem Ipsum, y más recientemente con software de autoedición, como por ejemplo Aldus PageMaker, el cual incluye versiones de Lorem Ipsum.

¿Por qué lo usamos?

Es un hecho establecido hace demasiado tiempo que un lector se distraerá con el contenido del texto de un sitio mientras que mira su diseño. El punto de usar Lorem Ipsum es que tiene una distribución más o menos normal de las letras, al contrario de usar textos como por ejemplo "Contenido aquí, contenido aquí". Estos textos hacen parecerlo un espacio que se puede leer. Muchos paquetes de autoedición y editores de páginas web usan el Lorem Ipsum como su texto por defecto, y al hacer una búsqueda de "Lorem Ipsum" va a dar por resultado muchos sitios

Tamaño en X: 450

Tamaño en Y: 350

+



Uso de librerías de interfaz gráfica

Creación de una ventana

Observa el siguiente ejemplo, donde se creó una ventana en la cual se modifican las dimensiones del **componente JTextArea**:

Para modificar las dimensiones del componente, hay que escribir su tamaño en X y en Y, y después, solo presionar la tecla *Enter*

Tamaño en X: 100

Tamaño en Y: 50

+

+

mezcló de tal manera que logró hacer un libro de textos especimen. No sólo sobrevivió 500 años, sino que también ingresó como texto de relleno en documentos electrónicos, quedando esencialmente igual al original. Fue popularizado en los 60s con la creación de las hojas "Letrasas", las cuales contenían pasajes de Lorem Ipsum, y más recientemente con software de autoedición, como por ejemplo Aldus PageMaker, el cual incluye versiones de Lorem Ipsum.

¿Por qué lo usamos?

Es un hecho establecido hace demasiado tiempo que un lector se distraerá con el contenido del texto de un sitio mientras que mira su diseño. El punto de usar Lorem Ipsum es que tiene una distribución más o menos normal de las letras, al contrario de usar textos como por ejemplo "Contenido aquí, contenido aquí". Estos textos hacen parecerlo un espacio que se puede leer. Muchos paquetes de autoedición y editores de páginas web usan el Lorem Ipsum como su texto por defecto, y al hacer una búsqueda de "Lorem Ipsum" va a dar por resultado muchos sitios

Tamaño en X: 450

Tamaño en Y: 350

+



Uso de librerías de interfaz gráfica

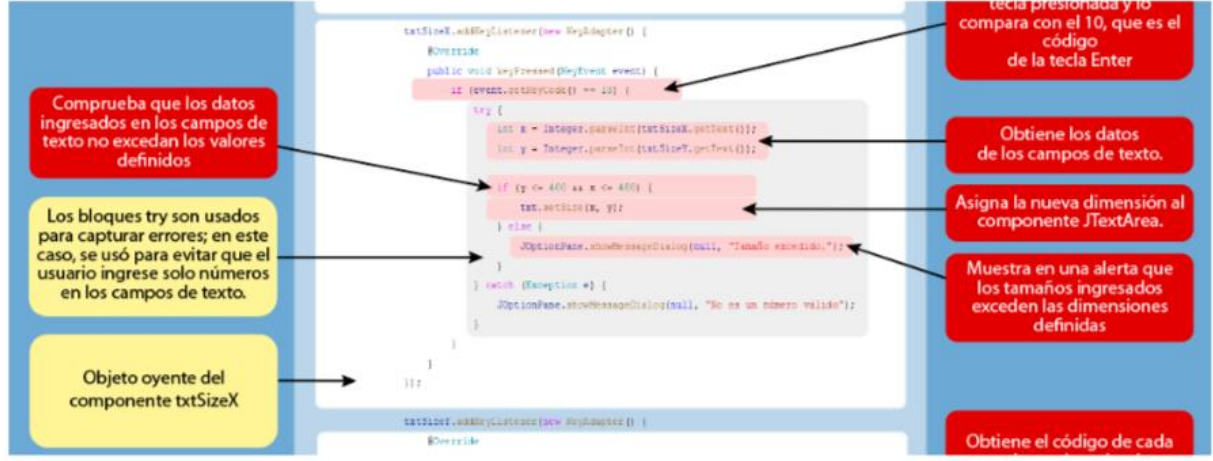
Creación de una ventana








Pulsa aquí para ver el PDF.



Comprueba que los datos ingresados en los campos de texto no excedan los valores definidos

Los bloques try son usados para capturar errores; en este caso, se usó para evitar que el usuario ingrese solo números en los campos de texto.

Objeto oyente del componente txtSizeX

```

txtSizeX.addActionListener(new KeyListener() {
    @Override
    public void keyPressed(KeyEvent event) {
        if (event.getKeyCode() == 10) {
            try {
                int x = Integer.parseInt(txtSizeX.getText());
                int y = Integer.parseInt(txtSizeY.getText());
                if (x <= 400 && y <= 400) {
                    txt.setSize(x, y);
                } else {
                    JOptionPane.showMessageDialog(null, "Tamaño excedido.");
                }
            } catch (Exception e) {
                JOptionPane.showMessageDialog(null, "No es un número válido.");
            }
        }
    }
});
txtSizeY.addActionListener(new KeyListener() {
    @Override
    // ...

```

tecla presionada y lo compara con el 10, que es el código de la tecla Enter

Obtiene los datos de los campos de texto.

Asigna la nueva dimensión al componente JTextArea.

Muestra en una alerta que los tamaños ingresados exceden las dimensiones definidas

Obtiene el código de cada

Creación de una ventana: https://drive.google.com/file/d/1kxI92SCU1DOmXVJno6tIZY_0YrhUiwyV/view



Uso de librerías de interfaz gráfica

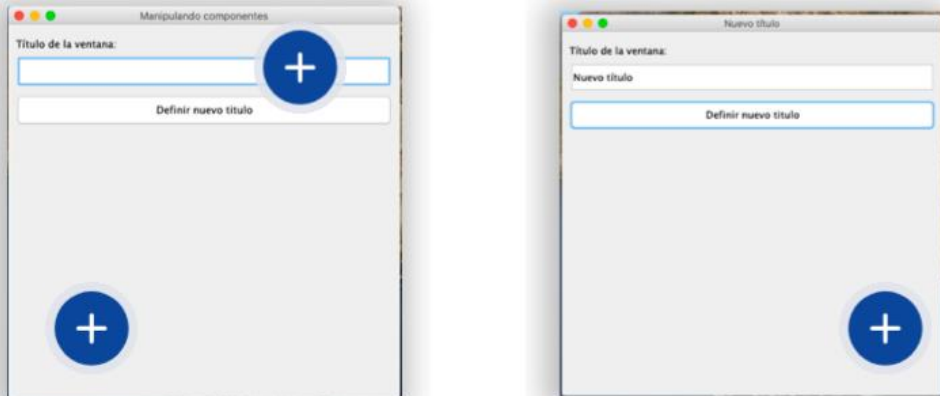
Modificación del título de una ventana






Observa el siguiente ejemplo donde se modifica el título de una ventana creada previamente.

- Se usa **JTextField** para ingresar el nuevo título
- Se usa un **botón** que se encarga de asignarlo a la ventana





Uso de librerías de interfaz gráfica

Modificación del título de una ventana

Campo de texto donde se ingresa el título de la ventana.

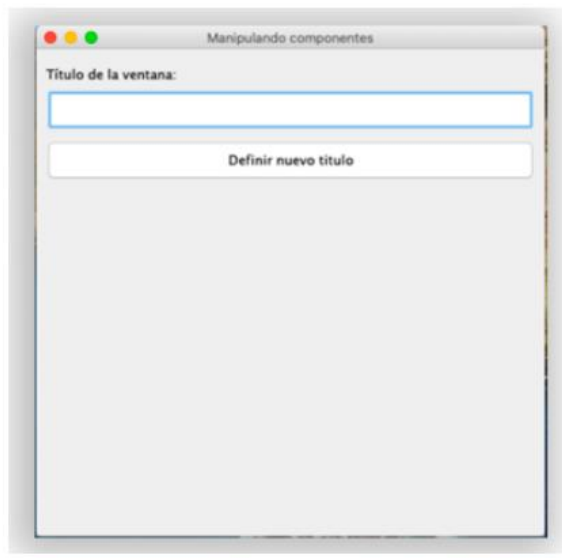


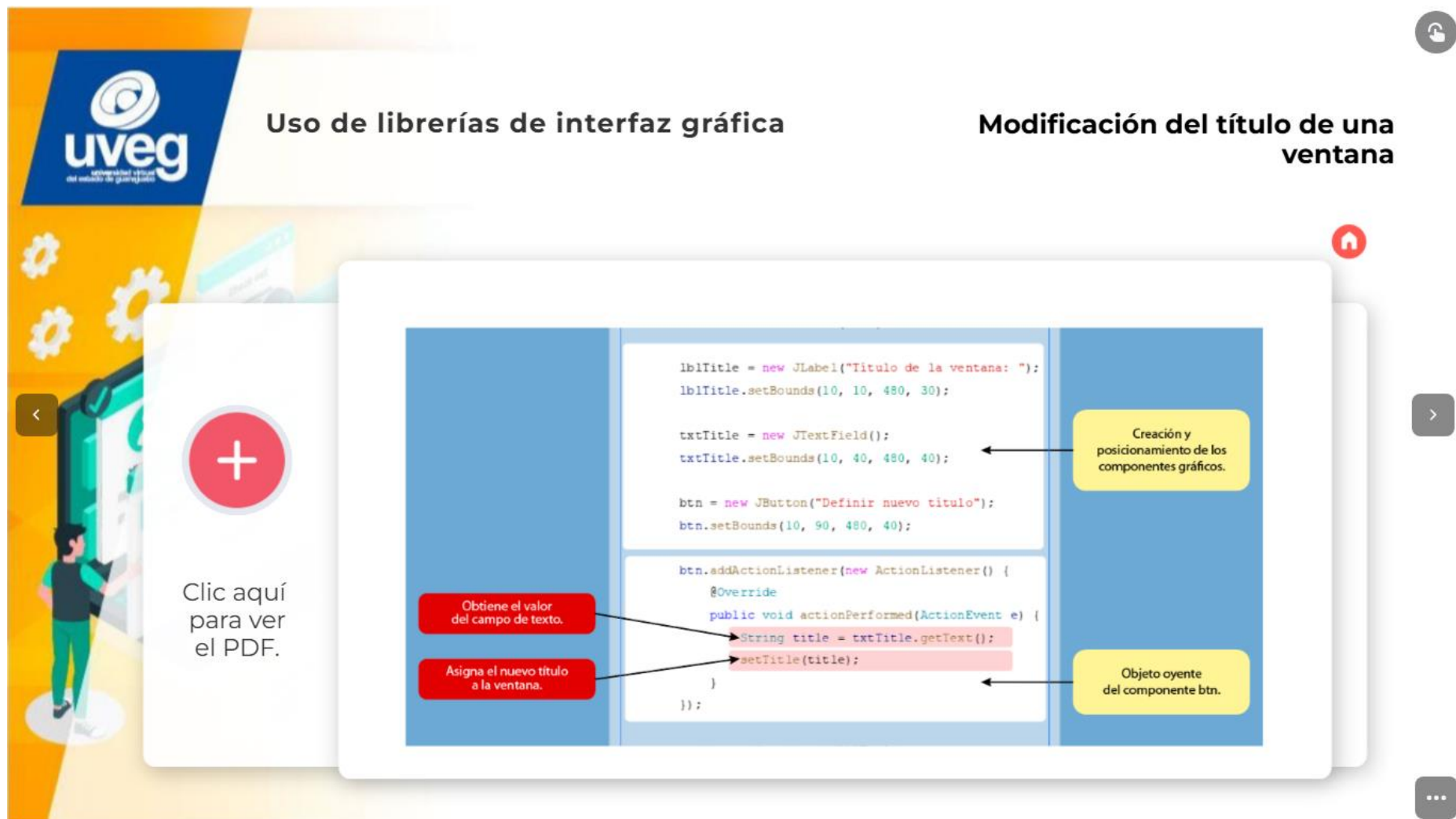
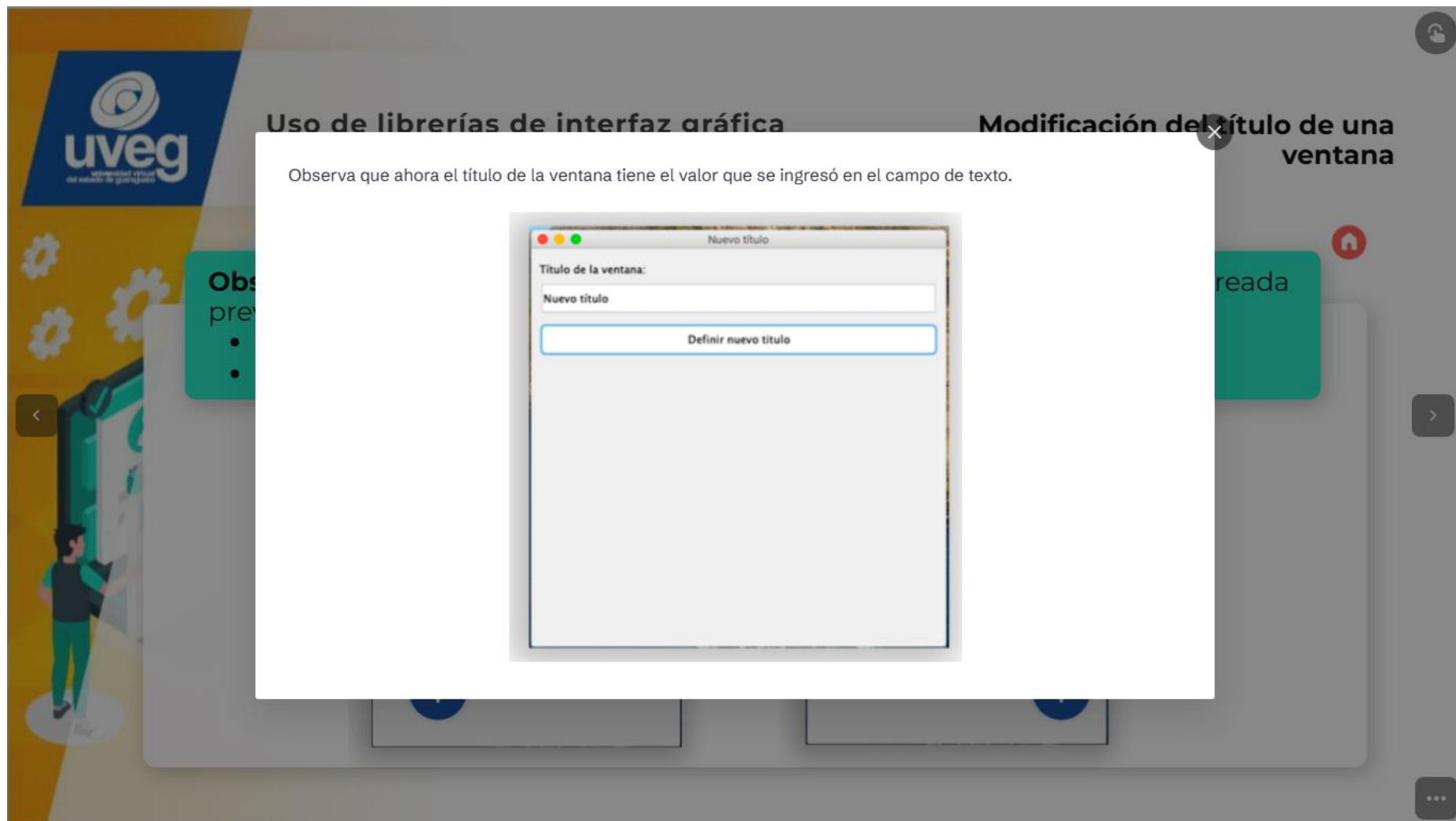


Uso de librerías de interfaz gráfica

Modificación del título de una ventana

Botón que asigna al título anterior de la ventana, el valor ingresado en el campo de texto.





Modificación del título de una ventana: <https://drive.google.com/file/d/150n0hfB5NSTY3snu7LtFZlcnIATk7dno/view>



Uso de librerías de interfaz gráfica

Creación de una ventana en la que, a partir de un botón, se pueden generar más botones

Observa el siguiente **ejemplo donde** se creó una **ventana** en la que, a partir de un **botón**, se pueden generar más botones de forma dinámica:

Manipulando componentes

Nuevo botón

+

Manipulando componentes

Nuevo botón

Nuevo 1	Nuevo 2	Nuevo 3
Nuevo 4	Nuevo 5	Nuevo 6
Nuevo 7	Nuevo 8	Nuevo 9
Nuevo 10	Nuevo 11	Nuevo 12
Nuevo 13	Nuevo 14	Nuevo 15
Nuevo 16	Nuevo 17	Nuevo 18
Nuevo 19	Nuevo 20	Nuevo 21

+



Uso de librerías de interfaz gráfica

Creación de una ventana en la que, a partir de un botón, se pueden generar más botones

La ventana comienza con solo un botón.

Manipulando componentes

Nuevo botón



Uso de librerías de interfaz gráfica

Creación de una ventana en la que, a partir de un botón, se pueden generar más botones

A medida que se presiona el botón Nuevo botón, se agregan uno más a la ventana.



Creación de una ventana en la que, a partir de un botón, se pueden generar más botones:

https://drive.google.com/file/d/1y8xkYIT2iCTzaB_CSxKty-71RAh0_WFg/view



Uso de librerías de interfaz gráfica

Librerías de interfaz gráfica

Antes de empezar a hablar de librerías de interfaz gráfica, es necesario definir qué es una **librería**.

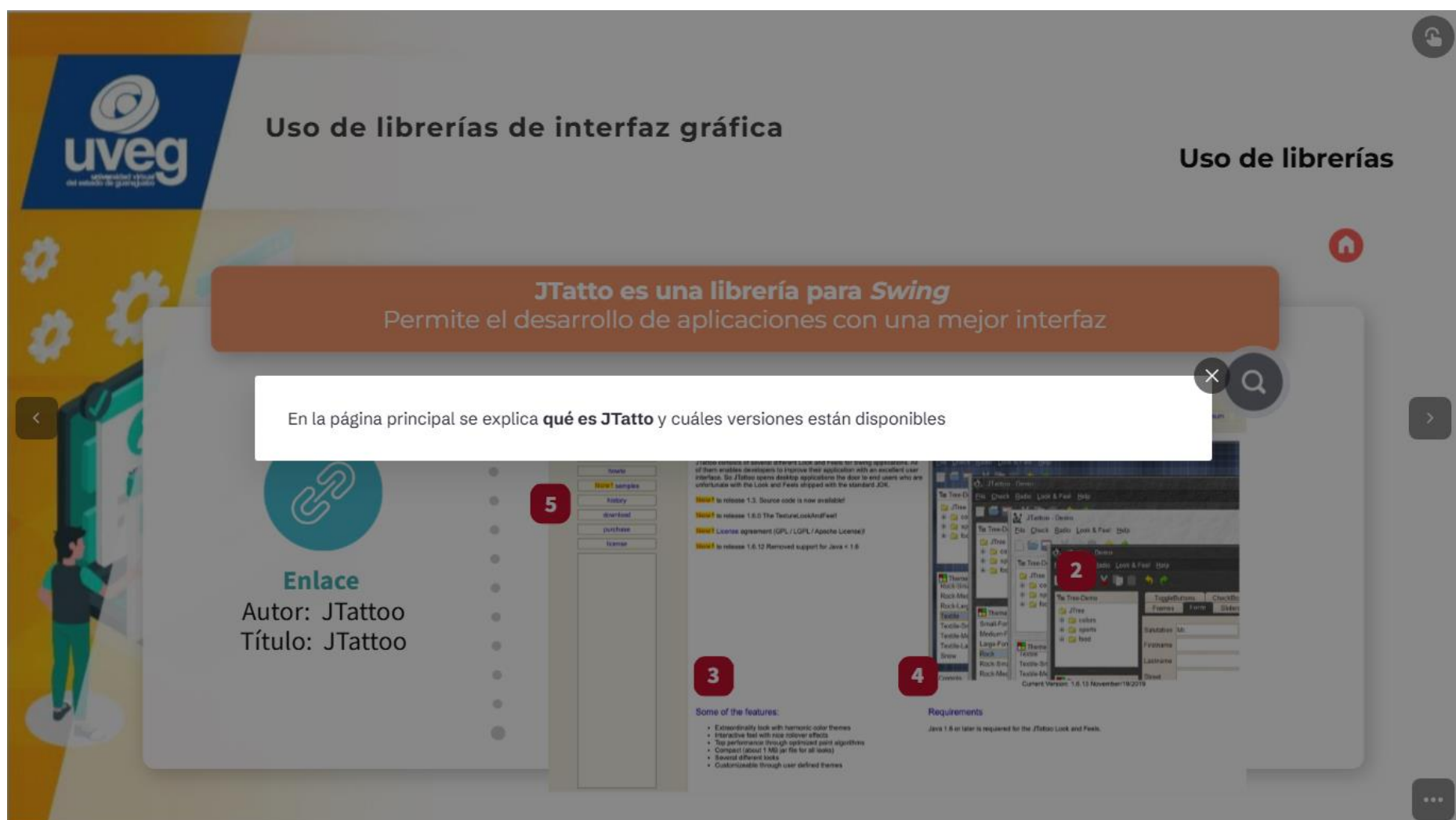
Una **librería en Java** puede definirse como un conjunto de clases agrupadas en un archivo **jar**.

1. Las clases contienen atributos y métodos propios.

2. Estas librerías permiten reducir los tiempos de desarrollo de una aplicación, lo que facilita la implementación de alguna función específica.



JTattoo: <https://jtattoo.de/>





Uso de librerías de interfaz gráfica

Uso de librerías

JTatto es una librería para *Swing*

Permite el desarrollo de aplicaciones con una mejor interfaz

Aquí se muestran los **diseños** que puedes usar en la aplicación



Enlace

Autor: JTattoo
Título: JTattoo

5

Download samples

history

download

purchase

license

3

Some of the features:

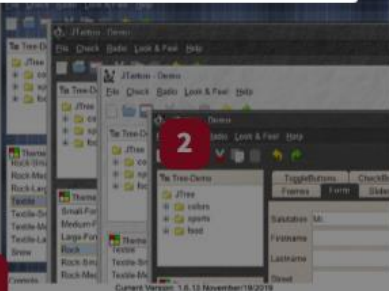
- Extraordinarily look with harmonic color themes
- Interactive feel with nice color effects
- Top performance through optimized paint algorithms
- Compact (about 1 MB jar file for all uses)
- Several different looks
- Customizable through user defined themes

4

Requirements

Java 1.8 or later is required for the JTattoo Look and Fools.

2





Uso de librerías de interfaz gráfica

Uso de librerías

JTatto es una librería para *Swing*

Permite el desarrollo de aplicaciones con una mejor interfaz

Aquí se muestran las **características de la librería**



Enlace

Autor: JTattoo
Título: JTattoo

5

Download samples

history

download

purchase

license

3

Some of the features:

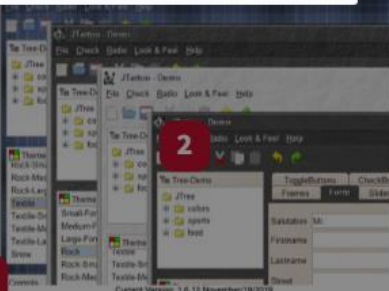
- Extraordinarily look with harmonic color themes
- Interactive feel with nice color effects
- Top performance through optimized paint algorithms
- Compact (about 1 MB jar file for all uses)
- Several different looks
- Customizable through user defined themes

4

Requirements

Java 1.8 or later is required for the JTattoo Look and Fools.

2





Uso de librerías de interfaz gráfica

Uso de librerías

JTatto es una librería para *Swing*

Permite el desarrollo de aplicaciones con una mejor interfaz

Especifica los **requerimientos mínimos** para poder usar la librería



Enlace

Autor: JTattoo
Título: JTattoo

5

download

3

Some of the features:

- Extraordinarily look with harmonic color themes
- Interactive feel with nice colorner effects
- Top performance through optimized paint algorithms
- Compact (about 1 MB jar file for all uses)
- Several different looks
- Customizable through user defined themes

4

Requirements

Java 1.8 or later is required for the JTattoo Look and Fools.

2

2



Uso de librerías de interfaz gráfica

Uso de librerías

JTatto es una librería para *Swing*

Permite el desarrollo de aplicaciones con una mejor interfaz

Para descargar la librería, pulsa en **download**



Enlace

Autor: JTattoo
Título: JTattoo

5

download

3

Some of the features:

- Extraordinarily look with harmonic color themes
- Interactive feel with nice colorner effects
- Top performance through optimized paint algorithms
- Compact (about 1 MB jar file for all uses)
- Several different looks
- Customizable through user defined themes

4

Requirements

Java 1.8 or later is required for the JTattoo Look and Fools.

2

2



Uso de librerías de interfaz gráfica

Descarga JTattoo

Pulsa sobre “The JTattoo jar file”, para que comience la descarga



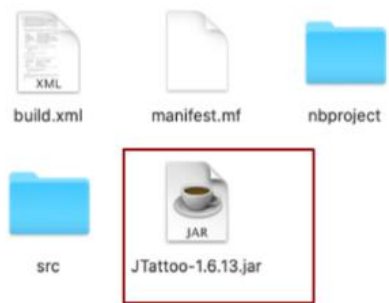
A continuación, te mostramos los pasos para la descarga:



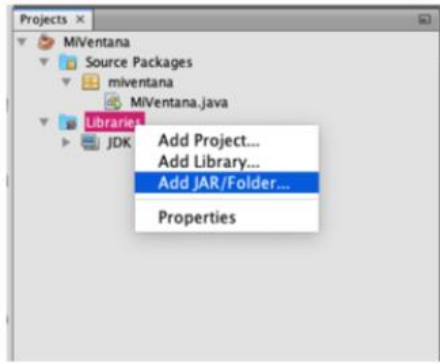
Uso de librerías de interfaz gráfica

Implementación JTattoo

Con el siguiente procedimiento, agregarás el archivo descargado a la raíz del proyecto de **NetBeans**:



En NetBeans, presiona el botón derecho del ratón o *touchpad* sobre **Libraries** y selecciona la opción Add **JAR/Folder...**



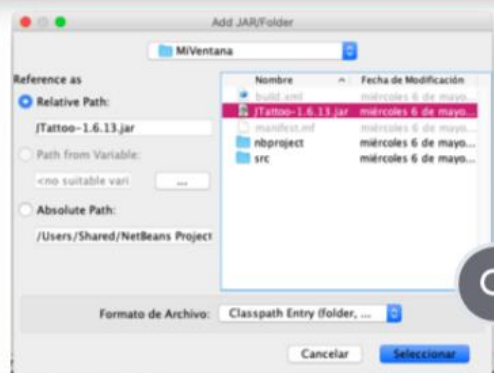


Uso de librerías de interfaz gráfica

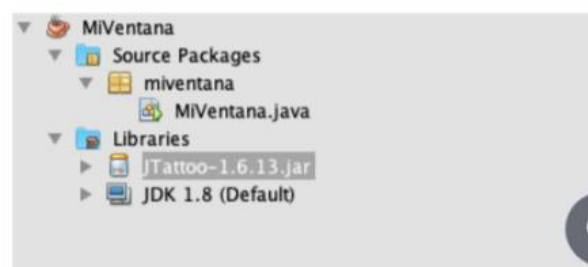
JTattoo



En la siguiente ventana, busca y selecciona la librería que descargaste antes, es decir, **JTattoo**:



Una vez agregada, debe aparecer la **librería JTattoo** en la carpeta **Libraries**



Uso de librerías de interfaz gráfica

En la clase principal de tu programa:

- Agrega el **siguiente fragmento de código** para que puedas usar la librería JTattoo.



1 El bloque **try-catch** se usa para capturar los posibles errores en el código escrito entre las llaves de la palabra reservada **try**; en este caso, si no existe el **LookAndFeel** de la librería JTattoo

```
import javax.swing.UIManager;

public class MiVentana {

    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel("com.jtattoo.plaf.smart.SmartLookAndFeel");
        } catch (Exception ex) {
            System.out.println(ex.getMessage());
        }
    }
}
```

2 Selecciona el **LookAndFeel** que provee la librería JTattoo. Si quieres ver un ejemplo de los **LookAndFeel**, presiona **aquí**.





Uso de librerías de interfaz gráfica

En la clase principal de tu programa:

- Agrega el **siguiente fragmento de código** para que puedas usar la librería JTattoo.

Recuerda

El término LookAndFeel hace referencia al diseño que tendrá una aplicación, ya sea, en botones, campos de texto, etcétera.

```
public static void main(String[] args) {  
    try {  
        UIManager.setLookAndFeel("com.jtattoo.plaf.smart.SmartLookAndFeel");  
    } catch (Exception ex) {  
        System.out.println(ex.getMessage());  
    }  
}
```

1

El bloque try-catch se usa para capturar los posibles errores en el código escrito entre las llaves de la palabra reservada try; en este caso, si no existe el LookAndFeel de la librería JTattoo

2

Selecciona el LookAndFeel que provee la librería JTattoo. Si quieres ver un ejemplo de los LookAndFeel, presiona [aquí](#).



Uso de librerías de interfaz gráfica

Los LookAndFeel's que proporciona la librería JTattoo

Observa los ejemplos de LookAndFeel's:



Librería: JTattoo

Ingresa un texto:

¡Hola mundo!

Mostrar Mensaje



Librería: JTattoo

Ingresa un texto:

¡Hola mundo!

Mostrar Mensaje

Para continuar visualizando el ejemplo, pulsa la flecha

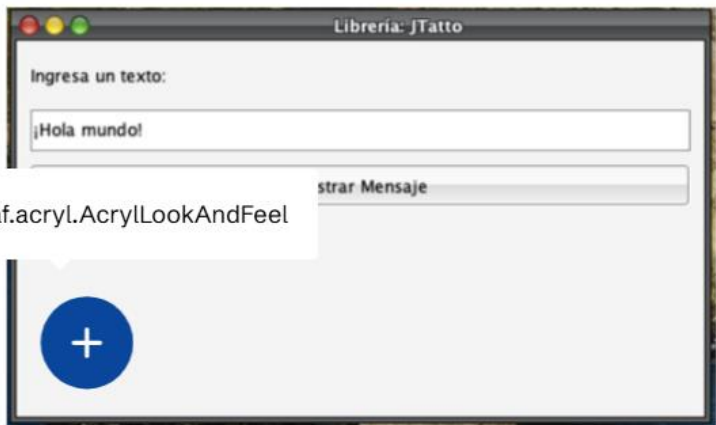


Uso de librerías de interfaz gráfica

Los LookAndFeel's que proporciona la librería JTattoo



Observa los ejemplos de LookAndFeel's:



com.jtattoo.plaf.acryl.AcrylLookAndFeel



com.jtattoo.plaf.hifi.HiFiLookAndFeel

Para continuar visualizando el ejemplo, pulsa la flecha



Uso de librerías de interfaz gráfica

Los LookAndFeel's que proporciona la librería JTattoo



Observa los ejemplos de LookAndFeel's:



com.jtattoo.plaf.acryl.AcrylLookAndFeel

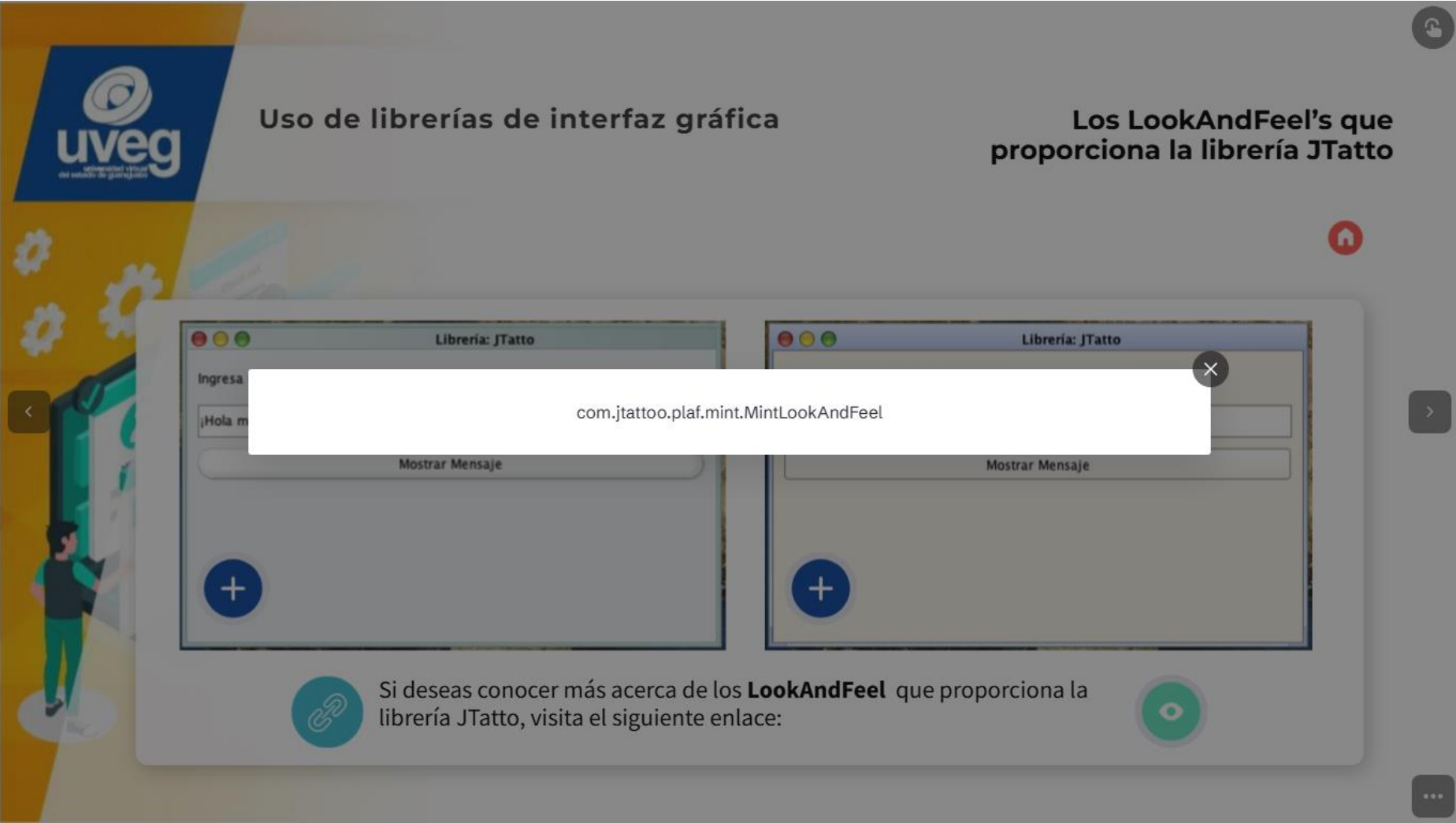


com.jtattoo.plaf.hifi.HiFiLookAndFeel

Para continuar visualizando el ejemplo, pulsa la flecha



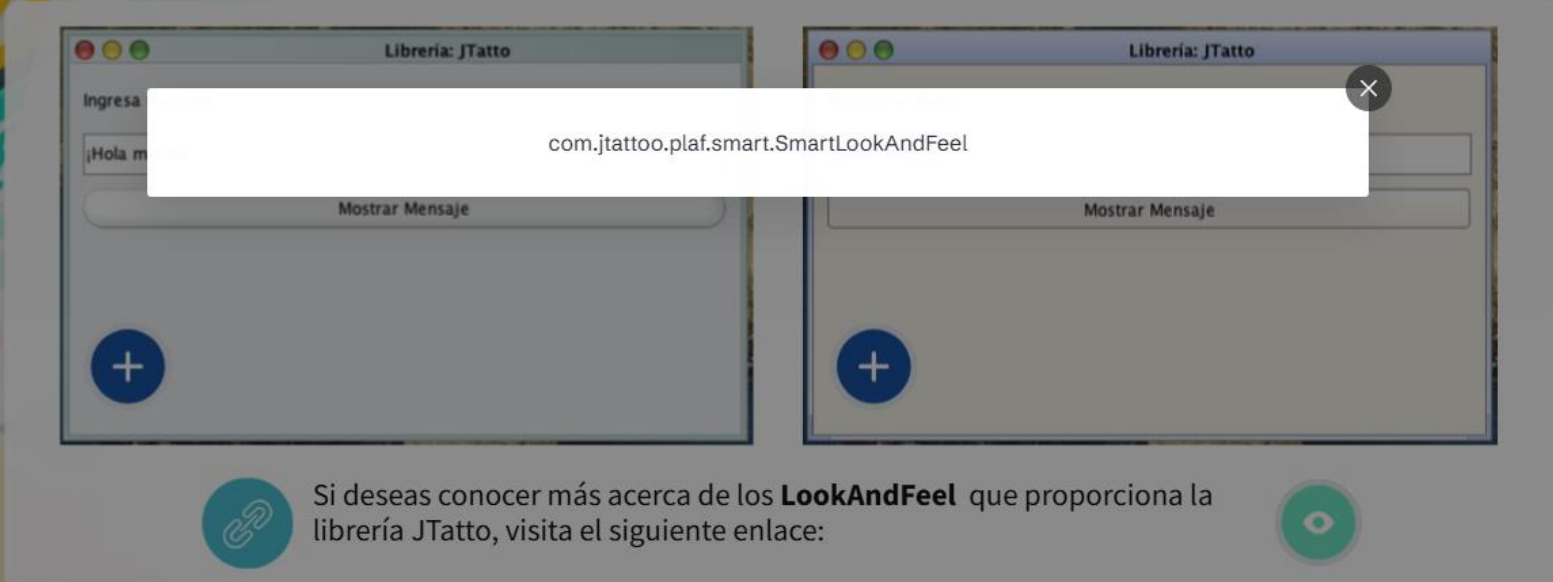
LEFT



RIGHT

 **Uso de librerías de interfaz gráfica**

Los LookAndFeel's que proporciona la librería JTatto



com.jtattoo.plaf.smart.SmartLookAndFeel

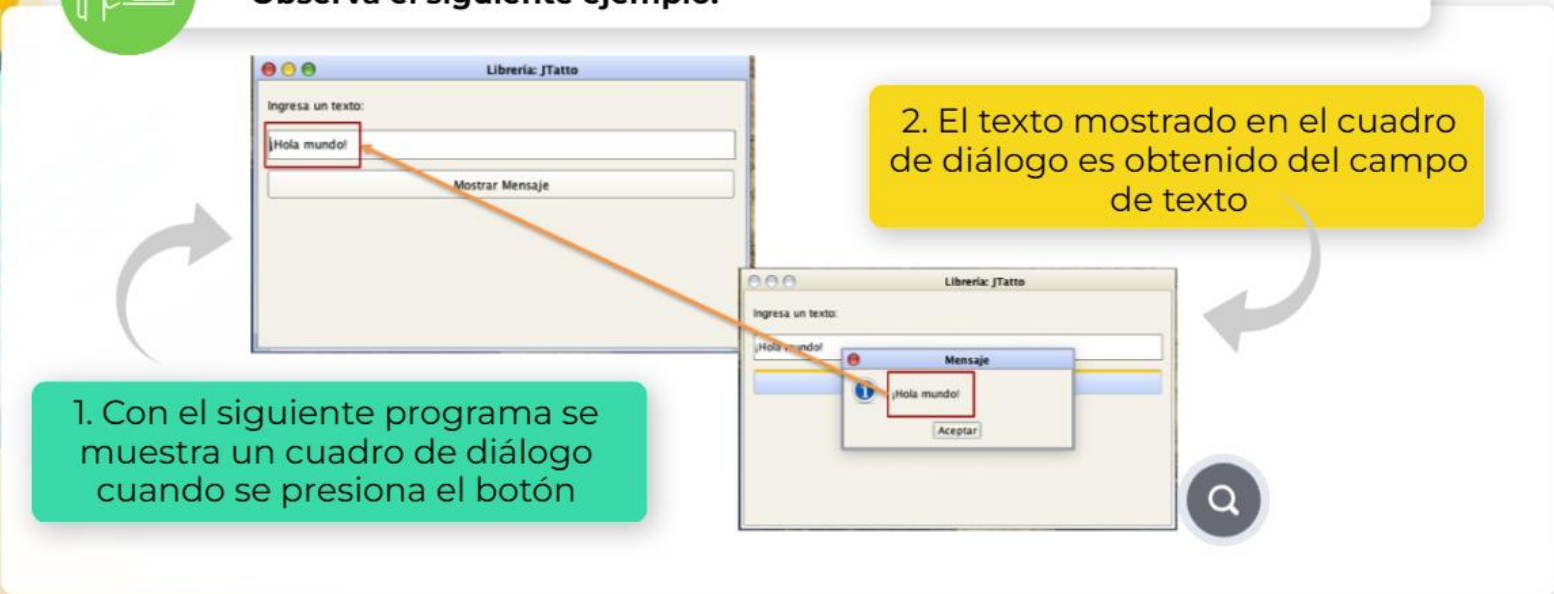
Si deseas conocer más acerca de los **LookAndFeel** que proporciona la librería JTatto, visita el siguiente enlace:

Enlace: <https://jtattoo.de/ScreenShots.html>

 **Uso de librerías de interfaz gráfica**

Utilizando la librería JTatto

 **Observa el siguiente ejemplo:**



1. Con el siguiente programa se muestra un cuadro de diálogo cuando se presiona el botón

2. El texto mostrado en el cuadro de diálogo es obtenido del campo de texto

Uso de librerías de interfaz gráfica

Escribe el LookAndFeel que
quieres utilizar

```
import javax.swing.JFrame;
import javax.swing.UIManager;

public class MiVentana {

    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel("com.jtattoo.plaf.smart.SmartLookAndFeel");

            Components components = new Components();
            components.setSize(500, 300);
            components.setLocationRelativeTo(null);
            components.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            components.setVisible(true);
        } catch (Exception ex) {
            System.out.println(ex.getMessage());
        }
    }
}
```

Crea una instancia de la clase Components

Escribe el LookAndFeel que
quieres utilizar

```
import javax.swing.JFrame;
import javax.swing.UIManager;

public class MiVentana {

    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel("com.jtattoo.plaf.smart.SmartLookAndFeel");

            Components components = new Components();
            components.setSize(500, 300);
            components.setLocationRelativeTo(null);
            components.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            components.setVisible(true);
        } catch (Exception ex) {
            System.out.println(ex.getMessage());
        }
    }
}
```

Crea una instancia de la clase Components



Uso de librerías de interfaz gráfica

Creación y ubicación de los componentes de interfaz gráfica

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextField;

public class Components extends JFrame {
    JLabel lbl;
    JTextField txt;
    JButton btn;

    public Components() {
        setTitle("Librería: Jtatto");
        setLayout(null);

        lbl = new JLabel("Ingresa un texto: ");
        lbl.setBounds(10, 10, 470, 30);

        txt = new JTextField("¡Hola mundo!");
        txt.setBounds(10, 50, 470, 30);

        btn = new JButton("Mostrar Mensaje");
        btn.setBounds(10, 90, 470, 30);
    }
}
```



Uso de librerías de interfaz gráfica

Agrega los componentes a la ventana

```
btn.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(null, txt.getText());
    }
});

getContentPane().add(lbl);
getContentPane().add(txt);
getContentPane().add(btn);
}
```

Objeto oyente del botón



Uso de librerías de interfaz gráfica

Manipulación de librerías

Hasta el momento has visto **cómo usar librerías creadas por un tercero**.

- Para esto, se desarrolló una clase que contiene cuatro métodos que realizan las operaciones básicas: sumar, restar, multiplicar y dividir.



Observa el siguiente ejemplo, ahí verás cómo crear y manipular una librería propia:

```
public class Operations {  
  
    public double addition(double addendA, double addendB) {  
        return addendA + addendB;  
    }  
  
    public double subtraction(double minuend, double subtrahend) {  
        return minuend - subtrahend;  
    }  
  
    public double multiplication(double factorA, double factorB) {  
        return factorA * factorB;  
    }  
  
    public double division(double dividend, double divisor) {  
        return dividend / divisor;  
    }  
}
```

Para continuar visualizando el ejemplo, pulsa la flecha



Uso de librerías de interfaz gráfica

Manipulación de librerías

```
public class Operations {  
  
    public double addition(double addendA, double addendB) {  
        return addendA + addendB;  
    }  
  
    public double subtraction(double minuend, double subtrahend) {  
        return minuend - subtrahend;  
    }  
  
    public double multiplication(double factorA, double factorB) {  
        return factorA * factorB;  
    }  
  
    public double division(double dividend, double divisor) {  
        return dividend / divisor;  
    }  
}
```

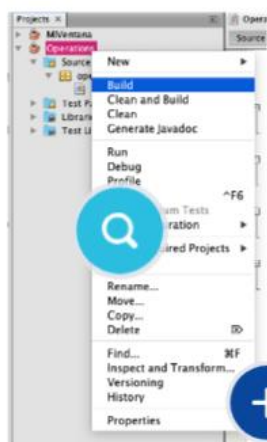
Para continuar visualizando el ejemplo, pulsa la flecha



Uso de librerías de interfaz gráfica



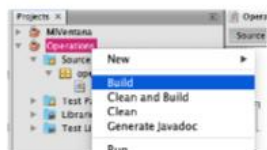
Observa detenidamente el ejemplo donde se presentan los pasos para crear y manipular una librería propia



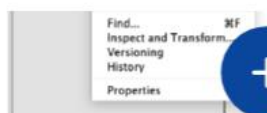
Uso de librerías de interfaz gráfica

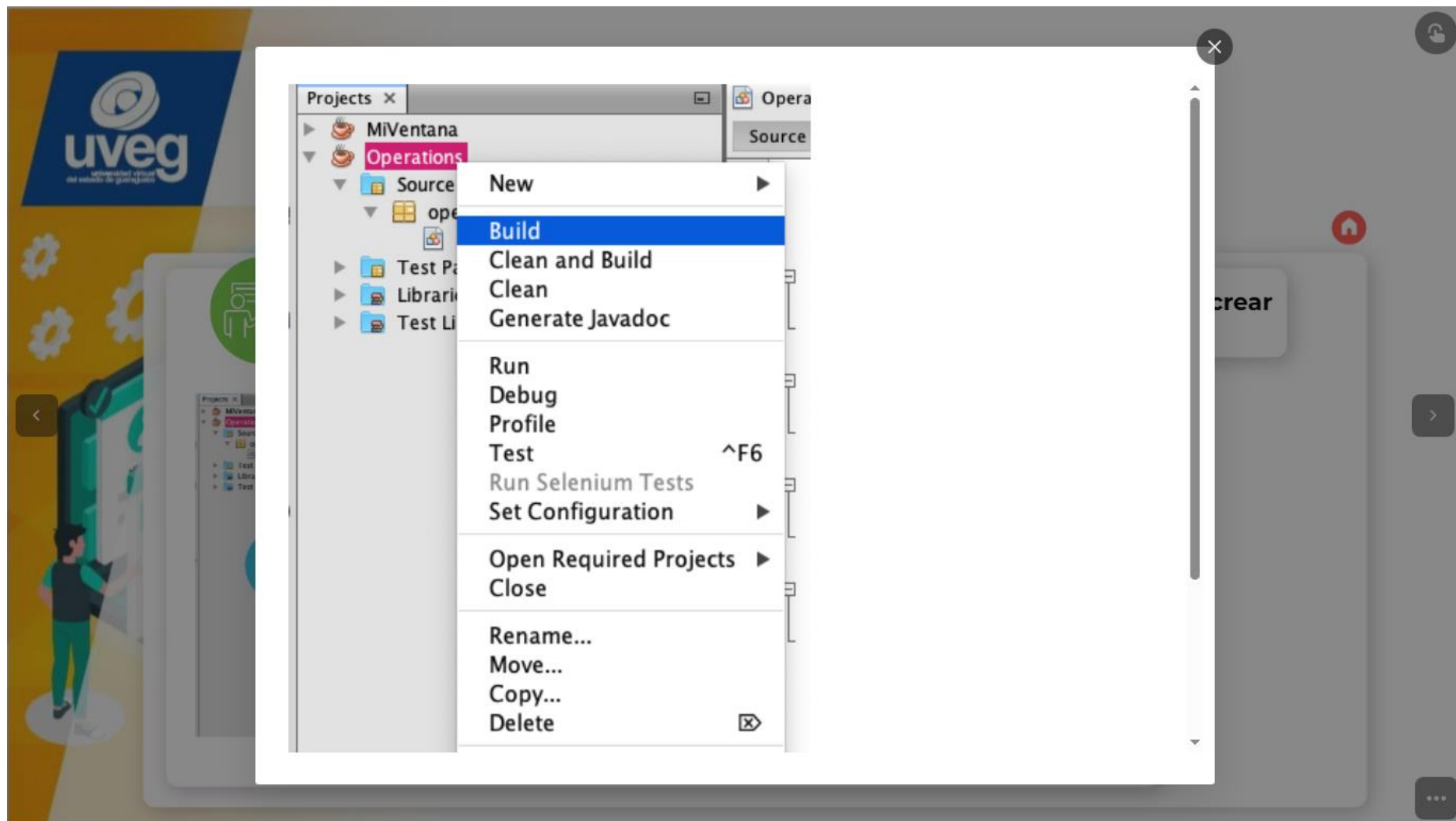


Observa detenidamente el ejemplo donde se presentan los pasos para crear y manipular una librería propia



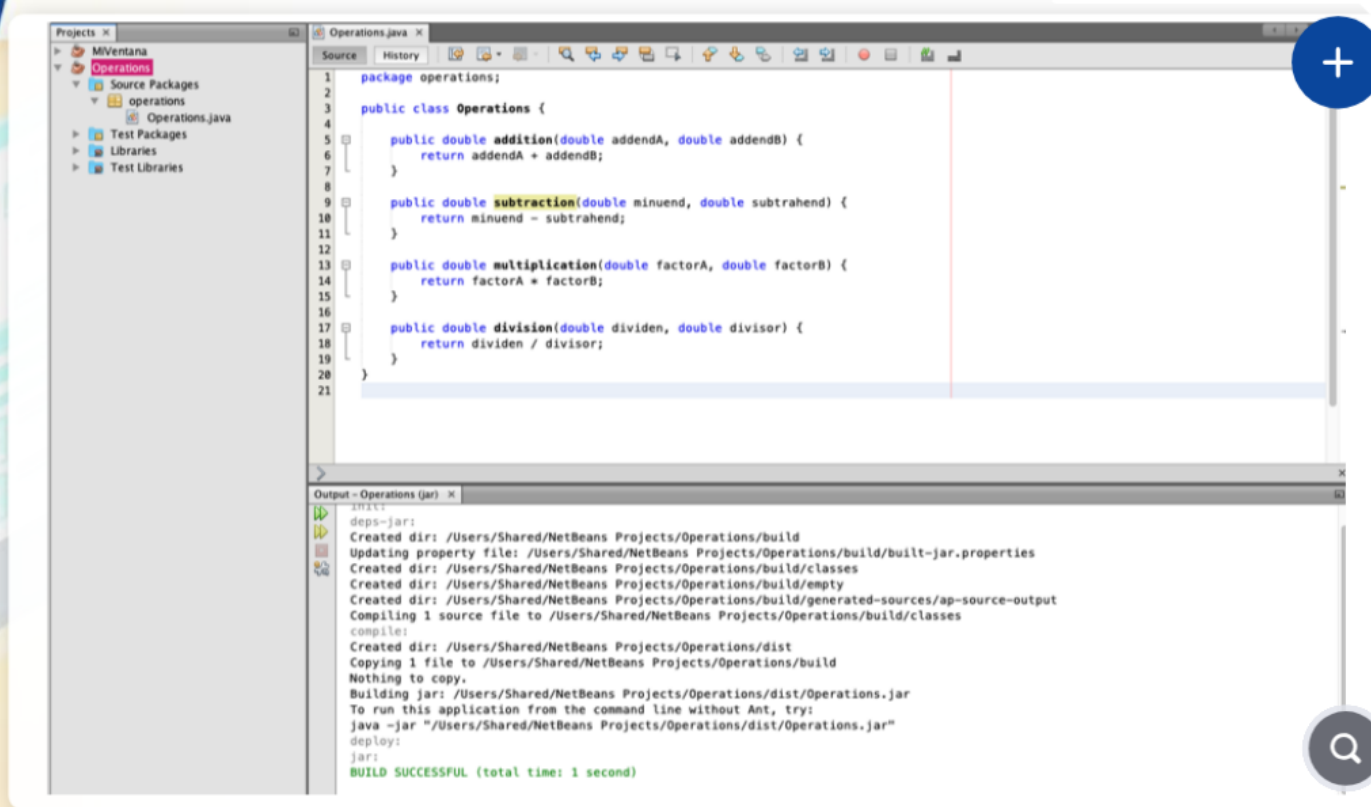
Para generar el archivo **jar** del proyecto, **presiona el botón derecho del ratón o touchpad**, sobre el nombre del proyecto y selecciona la opción **Build**.





Uso de librerías de interfaz gráfica

Una vez seleccionada la opción **Build**, la consola de NetBeans te mostrará la ruta donde se generó el archivo **jar**.



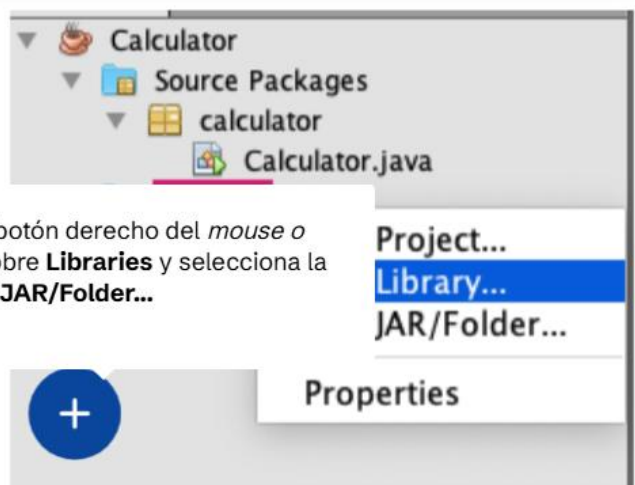
Pulsa aquí
para
continuar



Uso de librerías de interfaz gráfica

Observa los siguientes procedimientos

Crea un nuevo proyecto llamado Calculator y agrega el archivo jar que generaste antes



Presiona el botón derecho del *mouse* o *touchpad* sobre **Libraries** y selecciona la opción **Add JAR/Folder...**

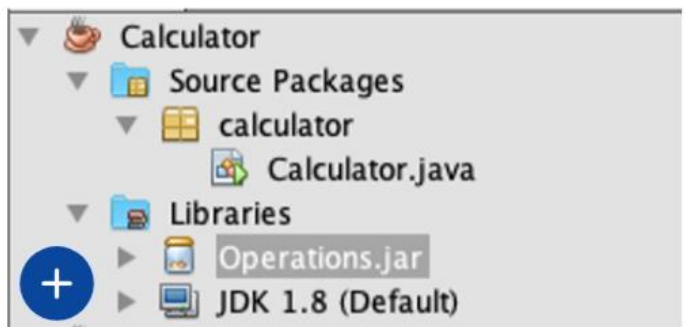
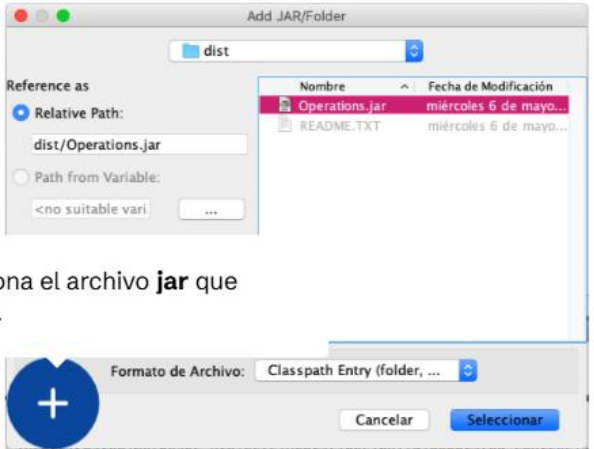
Project...
Library...
JAR/Folder...
Properties



Uso de librerías de interfaz gráfica

Observa los siguientes procedimientos

Busca y selecciona el archivo **jar** que generaste antes.

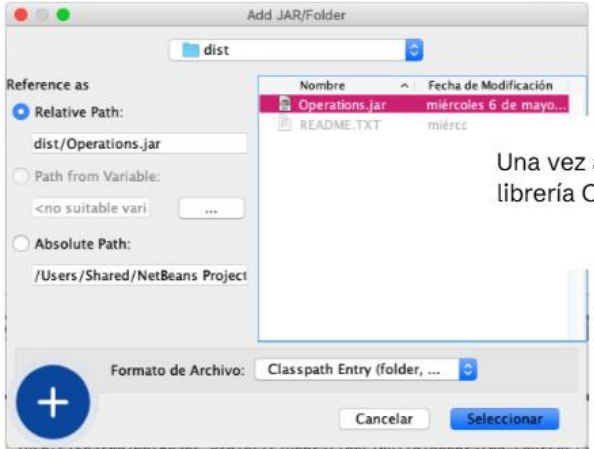




Uso de librerías de interfaz gráfica



Observa los siguientes procedimientos



Una vez agregado, debe aparecer la librería Operations en la carpeta Libraries.

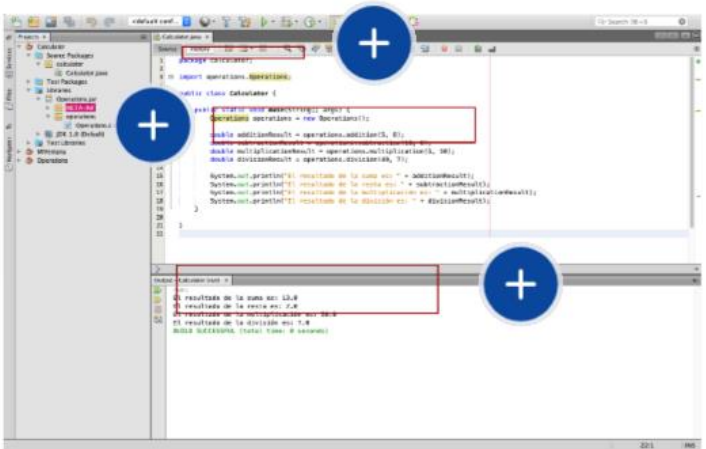




Uso de librerías de interfaz gráfica



Observa el siguiente ejemplo y sus resultados



Pulsa aquí para ampliar la imagen



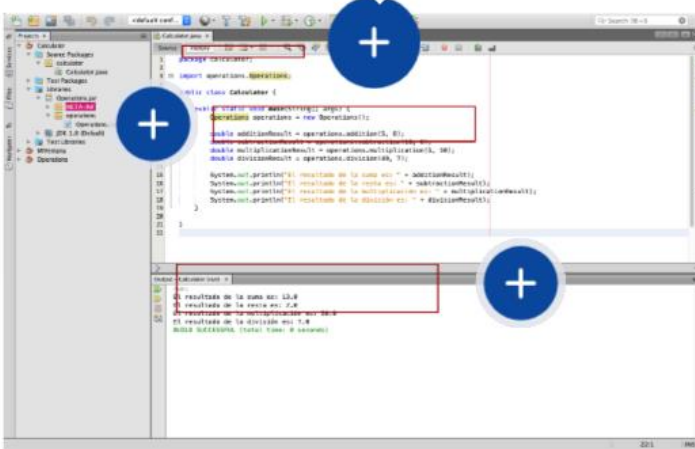
Uso de librerías de interfaz gráfica



Obs

Para importar la **clase Operations**, escribe el nombre del paquete (operations) seguido de un punto y enseguida, el nombre de la clase.

ultados





Pulsa aquí para ampliar la imagen

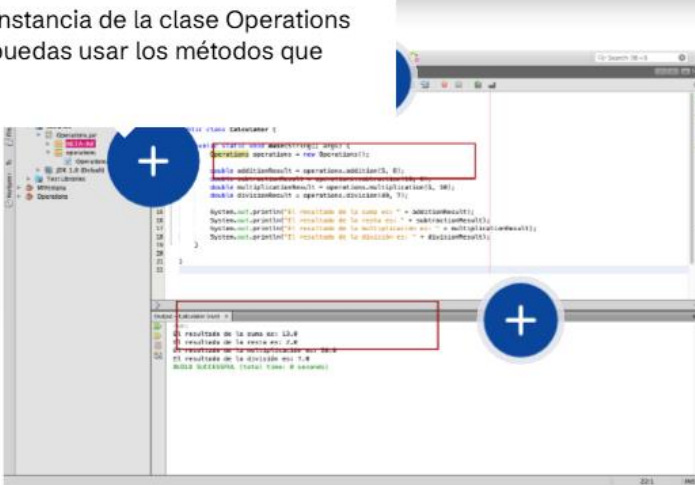


Uso de librerías de interfaz gráfica



Observa el siguiente ejemplo y sus resultados

Crea una instancia de la clase Operations para que puedas usar los métodos que contiene.





Pulsa aquí para ampliar la imagen

```
1 package calculator;
2
3 import operations.Operations;
4
5 public class Calculator {
6
7     public static void main(String[] args) {
8         Operations operations = new Operations();
9
10        double additionResult = operations.addition(5, 8);
11        double subtractionResult = operations.subtraction(10, 0);
12        double multiplicationResult = operations.multiplication(5, 10);
13        double divisionResult = operations.division(49, 7);
14
15        System.out.println("El resultado de la suma es: " + additionResult);
16        System.out.println("El resultado de la resta es: " + subtractionResult);
17        System.out.println("El resultado de la multiplicación es: " + multiplicationResult);
18        System.out.println("El resultado de la división es: " + divisionResult);
19    }
20
21 }
22
```

Output - Calculator (run) x

```
run:
El resultado de la suma es: 13.0
El resultado de la resta es: 2.0
El resultado de la multiplicación es: 50.0
El resultado de la división es: 7.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Uso de librerías de interfaz gráfica

En esta Lección aprendiste a:

- Conocer y usar los métodos que tienen los componentes gráficos para mejorar la experiencia de usuario.
- Observar cómo implementar una librería de interfaz gráfica, lo que mejora visualmente los componentes.
- Crear e implementar una librería propia.

Considera que con un **poco de práctica**, podrás **crear librerías** tan sencillas o complejas como desees.